دروس لغة بايثون

Python Lessons

من الأساسيات إلى البرمجة كائنية التوجه

16 درساً مع أمثلة وتمارين

فهرس الدروس

الدرس 1: دالة الطباعة والمتغيرات
الدرس 2: المُدخلات وتحويل الأنواع
الدرس 3: العمليات الحسابية
الدرس 4: الجمل الشرطية (if)
الدرس 5: الحلقة التكرارية for
الدرس 6: الحلقة while و break و continue
الدرس 7: التعامل مع النصوص (Strings)
الدرس 8: القوائم (Lists)
الدرس 9: المجموعات المرتبة (Tuples) والمجموعات (Sets)
الدرس 10: القواميس (Dictionaries)
الدرس 11: الدوال (Functions)
الدرس 12: الوحدات والمكتبات (import)
الدرس 13: معالجة الأخطاء (try / except)
الدرس 14: التعامل مع الملفات
الدرس 15: البرمجة كائنية التوجه (1): الأصناف والكائنات
الدرس 16: البرمجة كائنية التوجه (2): الوراثة

ماذا بعد الأساسيات؟

الدرس 1: دالة الطباعة والمتغيرات

تُستخدم دالة `print()` لعرض النصوص أو الأرقام أو نتائج العمليات الحسابية على الشاشة. نضع النص بين علامتي تنصيص داخل الأقواس.

```
print("مرحباً بك في عالم البرمجة")
```

المتغيرات (Variables)

المتغير مساحة في ذاكرة الكمبيوتر (يشبه صندوقاً) نخزّن فيها البيانات ونعطيها اسماً لنستدعيها به لاحقاً.

- **النص (str):** يُكتب بين علامتي تنصيص، مثل `"خالد"`
- **العدد الصحيح (int):** يُكتب مباشرة، مثل `30`
- **العدد العشري (float):** مثل `99.5`
- **القيمة المنطقية (bool):** إما `True` أو `False`

```
user_name = "خالد"
user_age = 30
price = 99.5
is_student = True

print(type(user_age))   # <class 'int'>
```

دمج النصوص مع المتغيرات

نفصل بين النص الثابت والمتغير بفاصلة `,` داخل `print`.

```
my_name = "محمد"
my_country = "الجزائر"

print("اسمي هو", my_name)
print("أنا أعيش في", my_country)
```

النتيجة على الشاشة:

```
اسمي هو محمد
أنا أعيش في الجزائر
```

قواعد تسمية المتغيرات

- يبدأ الاسم بحرف إنجليزي أو شرطة سفلية `_` وليس برقم.
- لا يحتوي على مسافات (استخدم `user_name` بدل `user name`).
- بايثون حسّاسة لحالة الأحرف: `Age` تختلف عن `age`.

تمرين

- خزّن اسمك ومدينتك وعمرك في ثلاثة متغيرات ثم اطبعها في ثلاثة أسطر.

الدرس 2: المُدخلات وتحويل الأنواع

عندما يصل بايثون إلى `input()` فإنه يوقف البرنامج مؤقتاً وينتظر المستخدم ليكتب شيئاً ويضغط Enter.

```
user_name = input("الرجاء إدخال اسمك: ")
print("مرحباً بك يا", user_name)
```

مشكلة الأرقام مع input()

أي شيء يكتبه المستخدم يعتبره بايثون **نصاً (str)** حتى لو كان رقماً. لذلك إذا حاولنا جمع نص مع رقم سيحدث خطأ من نوع `TypeError`. الحل هو التحويل باستخدام `int()` للأعداد الصحيحة أو `float()` للعشرية.

```
# الطريقة الخاطئة: age + 1 يسبب TypeError
age = input("كم عمرك؟ ")

# الطريقة الصحيحة: التحويل إلى رقم
age = int(input("كم عمرك؟ "))
print("العام القادم سيكون عمرك:", age + 1)
```

برنامج حاسبة مبسطة

```
num1 = int(input("أدخل الرقم الأول: "))
num2 = int(input("أدخل الرقم الثاني: "))

total = num1 + num2
print("مجموع الرقمين هو:", total)
```

💡 دوال التحويل الأكثر استخداماً: `int()` و`float()` و`str()`.

تمرين

- اطلب من المستخدم طوله بالسنتيمتر (رقم عشري) ثم اطبعه بالمتر.

الدرس 3: العمليات الحسابية

تدعم بايثون كل العمليات الحسابية، وفيها عمليات مفيدة مثل القسمة الصحيحة والباقي.

العملية	الرمز	مثال	الناتج
جمع	`+`	`17 + 5`	22
طرح	`-`	`17 - 5`	12
ضرب	`*`	`17 * 5`	85
قسمة	`/`	`17 / 5`	3.4
قسمة صحيحة	`//`	`17 // 5`	3
باقي القسمة	`%`	`17 % 5`	2
أُس	`**`	`17 ** 2`	289

```
a = 17
b = 5
print(a + b)    # 22
print(a / b)    # 3.4
print(a // b)   # 3
print(a % b)    # 2
print(a ** 2)   # 289
```

💡 العملية `%` مفيدة جداً: إذا كان `n % 2 == 0` فالعدد زوجي.

تمرين

- اطلب عدداً من المستخدم ثم اطبع مربعه ومكعبه.

الدرس 4: الجمل الشرطية (if)

في الحياة نأخذ قرارات بناءً على شروط، مثل: «إذا كان الجو ممطراً سآخذ المظلة». في بايثون نستخدم `if` لنفس الغرض.

علامات المقارنة

العلامة	المعنى
`==`	يساوي تماماً (علامتا يساوي وليس واحدة)
`!=`	لا يساوي
`>`	أكبر من
`<`	أصغر من
`>=`	أكبر من أو يساوي
`<=`	أصغر من أو يساوي

if و else

```
score = 40

if score >= 50:
    print("مبروك، لقد نجحت!")
else:
    print("للأسف، لم تنجح هذه المرة.")
```

💡 بعد النقطتين `:` يجب ترك مسافة بادئة (4 فراغات) قبل الأوامر التابعة للشرط. هذه المسافة هي التي تحدد ما يتبع الشرط.

الشروط المتعددة (elif)

```
age = 15

if age >= 18:
    print("أنت شخص بالغ.")
elif age >= 13:
    print("أنت في مرحلة المراهقة.")
else:
    print("أنت ما زلت طفلاً.")
```

النتيجة على الشاشة:

```
أنت في مرحلة المراهقة.
```

العوامل المنطقية: and / or / not

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("مسموح بالدخول")

if not has_id:
    print("أحضر هويتك")
```

- `and`: يجب تحقق الشرطين معاً.
- `or`: يكفي تحقق أحدهما.
- `not`: يعكس النتيجة.

تمرين

- اطلب درجة من المستخدم واطبع التقدير: ممتاز (90 فأكثر)، جيد جداً (80 فأكثر)، جيد (65 فأكثر)، مقبول (50 فأكثر)، وإلا راسب.

الدرس 5: الحلقة التكرارية for

نحتاج أحياناً تكرار أمر عدة مرات. بدلاً من كتابة الكود مئات المرات نستخدم الحلقات.

الدالة range()

- `range(5)` تولد: 0, 1, 2, 3, 4 (تبدأ من 0 وتتوقف قبل 5).
- `range(1, 6)` تولد: 1, 2, 3, 4, 5 (بداية ونهاية، والنهاية غير مشمولة).
- `range(0, 10, 2)` تولد: 0, 2, 4, 6, 8 (الرقم الثالث هو الخطوة).

```
for i in range(3):
    print("أنا أحب البرمجة")
```

النتيجة على الشاشة:

```
أنا أحب البرمجة
أنا أحب البرمجة
أنا أحب البرمجة
```

طباعة قيمة العداد

```
for number in range(1, 6):
    print(number)
```

النتيجة على الشاشة:

```
1
2
3
4
5
```

مثال: حساب المجموع

```
total = 0
for n in range(1, 6):
    total = total + n

print("المجموع:", total)
```

النتيجة على الشاشة:

```
المجموع: 15
```

تمرين

- اطبع جدول الضرب لعدد يدخله المستخدم (من 1 إلى 10).

الدرس 6: الحلقة while و break و continue

حلقة `while` تكرر الكود **ما دام الشرط صحيحاً**. نستخدمها عندما لا نعرف مسبقاً عدد مرات التكرار.

```
count = 1
while count <= 5:
    print("العدد:", count)
    count += 1
```

النتيجة على الشاشة:

```
العدد: 1
العدد: 2
العدد: 3
العدد: 4
العدد: 5
```

💡 إذا نسيت تغيير المتغير داخل الحلقة (هنا `count += 1`) فستصبح الحلقة لا نهائية. لإيقاف البرنامج اضغط Ctrl + C.

الأمر break: الخروج من الحلقة

```
while True:
    password = input("أدخل كلمة السر: ")
    if password == "1234":
        print("تم الدخول بنجاح")
        break
    print("كلمة السر خاطئة، حاول مرة أخرى")
```

الأمر continue: تخطي اللفة الحالية

```
for i in range(1, 8):
    if i % 2 == 0:
        continue
    print(i)
```

النتيجة على الشاشة:

```
1
3
5
7
```

💡 إذا استخدمت `while True` لتكرار عمل مستمر (مثل مراقبة شيء ما) فأضف `time.sleep(1)` داخلها بعد `im port time` حتى لا يستهلك البرنامج المعالج بالكامل.

تمرين

- اكتب لعبة تخمين: الرقم السري 7، ويستمر البرنامج بسؤال المستخدم حتى يخمّنه.

الدرس 7: التعامل مع النصوص (Strings)

النص في بايثون سلسلة من الحروف، ويمكن الوصول لأي حرف بترتيبه (يبدأ العد من 0).

```
text = "Python"
print(text[0])      # P
print(text[-1])     # n  (الأخير)
print(text[0:3])    # Pyt (من 0 حتى قبل 3)
print(len(text))    # 6
```

دوال النصوص الشائعة

```
print("python".upper())              # PYTHON
print("PYTHON".lower())              # python
print("  hello  ".strip())           # hello
print("a,b,c".split(","))            # ['a', 'b', 'c']
print("I like cats".replace("cats", "Python"))   # I like Python
print("Py" in "Python")              # True
```

تنسيق النصوص (f-strings)

أسهل طريقة لدمج المتغيرات داخل النص: نضع حرف `f` قبل علامة التنصيص، ونكتب المتغير بين `{ }`.

```
name = "سارة"
age = 22
print(f"اسمي {name} وعمري {age} سنة")
print(f"بعد 5 سنوات سيكون عمري {age + 5}")
```

النتيجة على الشاشة:

```
اسمي سارة وعمري 22 سنة
بعد 5 سنوات سيكون عمري 27
```

تمرين

- اطلب اسماً من المستخدم واطبعه بحروف كبيرة، ثم اطبع عدد حروفه.

الدرس 8: القوائم (Lists)

القائمة تخزّن عناصر متعددة في متغير واحد. تُنشأ بأقواس مربعة `[ ]` والعناصر مفصولة بفاصلة.

```
fruits = ["تفاح", "موز", "برتقال", "فراولة"]
print(fruits)
```

الوصول للعناصر (Index)

العد يبدأ من **0** وليس 1. العنصر الأول ترتيبه 0، والثاني 1، وهكذا. والترتيب `-1` يعني العنصر الأخير.

```
fruits = ["تفاح", "موز", "برتقال"]
print(fruits[0])    # تفاح
print(fruits[2])    # برتقال
print(fruits[-1])   # برتقال
```

التعديل والإضافة والحذف

```
names = ["أحمد", "خالد"]

names[0] = "محمد"    # تعديل
names.append("سارة")      # إضافة في النهاية
print(names)              # ['محمد', 'خالد', 'سارة']

names.remove("خالد")      # حذف بالقيمة
print(names)              # ['محمد', 'سارة']

numbers = [5, 2, 9, 1]
numbers.sort()
print(numbers)            # [1, 2, 5, 9]

last = numbers.pop()      # يحذف الأخير ويعيده
print(last)               # 9
print(len(numbers))       # 3
```

المرور على القائمة بحلقة for

```
cars = ["تويوتا", "فورد", "مرسيدس"]

for car in cars:
    print("أنا أحب سيارة:", car)
```

النتيجة على الشاشة:

```
أنا أحب سيارة: تويوتا
أنا أحب سيارة: فورد
أنا أحب سيارة: مرسيدس
```

تمرين

- اطلب من المستخدم إدخال 5 أرقام وخزّنها في قائمة، ثم اطبع أكبرها باستخدام `max()` ومجموعها باستخدام `sum()`.

الدرس 9: المجموعات المرتبة (Tuples) والمجموعات (Sets)

الـ Tuple

تشبه القائمة لكن **لا يمكن تعديلها** بعد إنشائها. تُكتب بأقواس عادية `( )`. مناسبة للبيانات الثابتة مثل الإحداثيات.

```
point = (10, 20)
print(point[0])      # 10

x, y = point         # فك القيم إلى متغيرات
print(x, y)          # 10 20
```

الـ Set

مجموعة من العناصر **بدون تكرار** وبدون ترتيب ثابت. تُكتب بأقواس متموجة `{ }`.

```
colors = {"أحمر","أخضر","أحمر"}
print(len(colors))        # 2 (حُذف المكرر)

colors.add("أزرق")
print("أخضر" in colors)   # True

numbers = [1, 2, 2, 3, 3, 3]
unique = sorted(set(numbers))
print(unique)             # [1, 2, 3]
```

💡 استخدم الـ set عندما تريد حذف التكرار أو التحقق السريع من وجود عنصر.

تمرين

- لديك القائمة `[4, 7, 4, 9, 7, 1]`. اطبع الأرقام الفريدة فقط مرتبة.

الدرس 10: القواميس (Dictionaries)

القاموس يخزّن البيانات على شكل **مفتاح: قيمة**، مثل ربط الاسم بأحمد والعمر بـ 28. يُنشأ بأقواس متموجة.

```
employee = {
    "name": "أحمد",
    "age": 28,
    "job": "مبرمج"
}
print(employee)
```

الوصول والتعديل والإضافة

```
print(employee["name"])     # أحمد

employee["age"] = 29                    # تعديل
employee["city"] = "سطيف"       # إضافة مفتاح جديد

print(employee.get("salary", "غير موجود"))    # غير موجود
```

💡 الوصول لمفتاح غير موجود بـ `[ ]` يسبب `KeyError`، أما `.get()` فتعيد قيمة افتراضية بدلاً من الخطأ.

المرور على القاموس

```
for key, value in employee.items():
    print(key, ":", value)
```

النتيجة على الشاشة:

```
name : أحمد
age : 29
job : مبرمج
city : سطيف
```

الفرق بين القائمة والقاموس

وجه المقارنة	القائمة (List)	القاموس (Dictionary)
الأقواس	مربعة `[ ]`	متموجة `{ }`
تنظيم البيانات	عناصر متتالية	أزواج مفتاح: قيمة
الوصول للعنصر	بالترتيب `list[0]`	بالمفتاح `dict["name"]`

تمرين

- أنشئ قاموساً لكتاب (العنوان، المؤلف، سنة النشر) ثم اطبع كل معلوماته بحلقة.

الدرس 11: الدوال (Functions)

إذا احتجت تكرار كود في أماكن مختلفة فاجمعه في «صندوق» له اسم وناده عند الحاجة. هذا الصندوق هو الدالة. نعرّفها بـ `def`.

```
def greet():
    print("أهلاً بك!")
    print("نتمنى لك يوماً سعيداً.")

greet()     # استدعاء الدالة
```

💡 الكود داخل الدالة لا يعمل إلا عند استدعائها باسمها مع القوسين.

المعاملات (Parameters)

```
def greet_user(username):
    print("مرحباً يا", username)

greet_user("صالح")
greet_user("فاطمة")
```

النتيجة على الشاشة:

```
مرحباً يا صالح
مرحباً يا فاطمة
```

إرجاع القيم (return)

أحياناً نريد من الدالة أن تحسب شيئاً وتعيد الناتج لنستخدمه في مكان آخر، بدلاً من طباعته مباشرة.

```
def add_numbers(num1, num2):
    result = num1 + num2
    return result

total = add_numbers(5, 10)
print("مجموع الرقمين هو:", total)
```

النتيجة على الشاشة:

```
مجموع الرقمين هو: 15
```

القيم الافتراضية

```
def power(base, exp=2):
    return base ** exp

print(power(5))      # 25
print(power(2, 3))   # 8
```

نطاق المتغيرات

المتغير المعرَّف داخل الدالة يوجد داخلها فقط، ولا يمكن استخدامه خارجها.

```
def show():
    secret = 10
    print(secret)

show()           # 10
# print(secret)  # خطأ NameError
```

تمرين

- اكتب دالة `is_even(n)` تعيد `True` إذا كان العدد زوجياً وإلا `False`.

الدرس 12: الوحدات والمكتبات (import)

بايثون تأتي مع مكتبات جاهزة (Modules) نستدعيها بـ `import`. وهناك مكتبات خارجية تُثبّت بالأمر `pip install اسم_المكتبة` في موجه الأوامر.

```
import math
import random

print(math.sqrt(16))          # 4.0
print(math.pi)                # 3.14159...
print(random.randint(1, 10)) # رقم عشوائي من 1 إلى 10
```

استيراد جزء من المكتبة

```
from math import sqrt

print(sqrt(25))   # 5.0
```

مكتبات قياسية مفيدة

- `math`: دوال رياضية.
- `random`: الأرقام والاختيارات العشوائية.
- `datetime`: التواريخ والأوقات.
- `time`: الانتظار `time.sleep(2)` وقياس الوقت.
- `json`: التعامل مع ملفات JSON.
- `os`: التعامل مع نظام التشغيل والمجلدات.

تمرين

- اكتب برنامجاً يختار رقماً عشوائياً بين 1 و 100 ويطبعه.

الدرس 13: معالجة الأخطاء (try / except)

إذا كتب المستخدم «عشرين» بدل 20 فستفشل `int()` وينهار البرنامج. لحماية البرنامج نستخدم `try` و`except`.

- **try:** نضع فيها الكود الذي قد يحدث فيه خطأ.
- **except:** الكود البديل الذي يعمل عند حدوث الخطأ بدلاً من انهيار البرنامج.

```
try:
    age = int(input("الرجاء إدخال عمرك بالأرقام: "))
    print("تم قبول عمرك:", age)
except ValueError:
    print("خطأ! اكتب أرقاماً صحيحة فقط (مثال: 25)")
```

💡 الأفضل دائماً تحديد نوع الخطأ (مثل `ValueError`) بدل كتابة `except:` وحدها، لأن `except:` العامة تخفي أخطاء أخرى لا تقصدها.

التعامل مع أخطاء محددة

```
try:
    num1 = int(input("أدخل البسط: "))
    num2 = int(input("أدخل المقام: "))
    print("ناتج القسمة هو:", num1 / num2)
except ZeroDivisionError:
    print("خطأ رياضي! لا يمكن القسمة على الصفر")
except ValueError:
    print("خطأ في المدخلات: يجب إدخال أرقام فقط!")
```

else و finally

- `else`: تعمل إذا **لم** يحدث أي خطأ.
- `finally`: تعمل **دائماً** سواء حدث خطأ أم لا (مفيدة لإغلاق الملفات مثلاً).

تكرار السؤال حتى يُدخل المستخدم قيمة صحيحة

```
while True:
    try:
        age = int(input("أدخل عمرك: "))
        break
    except ValueError:
        print("أدخل رقماً صحيحاً فقط")

print("عمرك:", age)
```

تمرين

- اطلب رقمين واقسم الأول على الثاني مع معالجة الخطأين: القسمة على صفر والإدخال غير الرقمي.

الدرس 14: التعامل مع الملفات

نستخدم `open()` لقراءة الملفات والكتابة فيها. الأفضل استخدامها مع `with` لأنها تغلق الملف تلقائياً.

الوضع	المعنى
`"r"`	قراءة (الافتراضي)
`"w"`	كتابة (يمسح محتوى الملف القديم)
`"a"`	إضافة في نهاية الملف

الكتابة في ملف

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("السطر الأول\n")
    f.write("السطر الثاني\n")
```

القراءة من ملف

```
with open("notes.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

النتيجة على الشاشة:

```
السطر الأول
السطر الثاني
```

💡 اكتب `encoding="utf-8"` دائماً عند التعامل مع النصوص العربية حتى لا تظهر الحروف مشوّهة.

ملفات JSON

```
import json

data = {"name": "أحمد", "age": 28}

with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False)

with open("data.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)

print(loaded["name"])   # أحمد
```

تمرين

- اكتب برنامجاً يطلب من المستخدم ملاحظة ويضيفها في نهاية ملف `notes.txt` ثم يعرض كل الملاحظات.

الدرس 15: البرمجة كائنية التوجه (1): الأصناف والكائنات

عندما تكبر البرامج يصعب تنظيمها بالمتغيرات والدوال فقط. تحوّل OOP عناصر الحياة الحقيقية إلى كود عبر مفهومين:

- **الفئة (Class):** المخطط أو القالب، مثل مخطط بناء سيارة.
- **الكائن (Object):** الشيء الحقيقي المصنوع من المخطط، مثل السيارة نفسها.

```
class Dog:
    breed = "هاسكي"
    color = "أبيض"

my_dog = Dog()
print("نوع الكلب:", my_dog.breed)
print("لون الكلب:", my_dog.color)
```

دالة البناء __init__

لجعل كل كائن فريداً نستخدم `__init__` التي تعمل تلقائياً عند إنشاء الكائن. الكلمة `self` تعني «هذا الكائن الحالي».

```
class Car:
    def __init__(self, brand, model_year):
        self.brand = brand
        self.model_year = model_year

car1 = Car("تويوتا", 2022)
car2 = Car("مرسيدس", 2025)

print("السيارة الأولى:", car1.brand)
print("موديل الثانية:", car2.model_year)
```

الدوال داخل الفئة (Methods)

```
class Student:
    def __init__(self, name):
        self.name = name

    def study(self):
        print(self.name, "يذاكر دروس بايثون الآن!")

student1 = Student("عمر")
student1.study()
```

النتيجة على الشاشة:

```
عمر يذاكر دروس بايثون الآن!
```

مثال: حساب بنكي

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"تم إيداع {amount}. الرصيد: {self.balance}")

account = BankAccount("ليلى", 1000)
account.deposit(500)
```

النتيجة على الشاشة:

```
تم إيداع 500. الرصيد: 1500
```

تمرين

- أضف للحساب البنكي دالة `withdraw(amount)` تسحب المبلغ، وتطبع رسالة خطأ إذا كان الرصيد غير كافٍ.

الدرس 16: البرمجة كائنية التوجه (2): الوراثة

الوراثة تتيح لفئة جديدة (فرعية) أن ترث خصائص ودوال فئة موجودة (أساسية) مع تضيف عليها أو تعدّلها، فلا نكرر الكود.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(self.name, "يصدر صوتاً")

class Dog(Animal):
    def speak(self):
        print(self.name, "ينبح: هاو هاو")

class Cat(Animal):
    def speak(self):
        print(self.name, "يموء: مياو")

animals = [Dog("ريكس"), Cat("لولو"), Animal("حيوان")]
for a in animals:
    a.speak()
```

النتيجة على الشاشة:

```
ريكس ينبح: هاو هاو
لولو يموء: مياو
حيوان يصدر صوتاً
```

هذا يسمى **تعدد الأشكال (Polymorphism)**: نفس الاستدعاء `speak()` يعطي سلوكاً مختلفاً حسب نوع الكائن.

استدعاء دالة البناء الأصلية بـ `super()`

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, grade):
        super().__init__(name)
        self.grade = grade

s = Student("سامي", 85)
print(s.name, s.grade)   # سامي 85
```

تمرين

- أنشئ فئة `Shape` وفئتين فرعيتين `Rectangle` و`Circle` لكل منهما دالة `area()` تحسب المساحة.

ماذا بعد الأساسيات؟

بعد إتقان الدروس السابقة، أنجز هذه المشاريع بنفسك دون نسخ الحلول، فالبرمجة تُتقن بالتطبيق:

- **لعبة تخمين الأرقام:** `random` و`while` و`if`.
- **حاسبة بقائمة اختيارات:** دوال وشروط ومعالجة أخطاء.
- **تطبيق إدارة المهام:** قوائم وقواميس وحفظ في ملف JSON.
- **نظام حسابات بنكية:** فئات ووراثة.

وبعدها اختر مساراً حسب هدفك:

- **تحليل البيانات:** `pandas` و`matplotlib`.
- **تطوير الويب:** `Flask` أو `Django`.
- **الأتمتة:** `requests` و`selenium` والتعامل مع الملفات.
- **مواضيع متقدمة:** المولّدات (generators)، المزخرفات (decorators)، الاختبارات `pytest`، البرمجة غير المتزامنة `asyncio`.